

Security Architecture Design: Internship Progress Dashboard

This document outlines the comprehensive security architecture for the "Internship Progress Dashboard" application. The goal is to build a highly secure, production-ready system that protects user data and prevents common web vulnerabilities using industry-standard best practices.

1. System Overview and Roles

The application is a dashboard for tracking intern progress, featuring task management and reporting.

Role	Description	Access Level
User (Student)	Manages and tracks their own tasks and progress.	Basic access (Self-view, update own tasks)
Admin (Mentor/Admin)	Oversees all users, manages tasks, and views all progress.	Full access (View all data, CRUD on tasks and users)

2. Authentication and Authorization (AuthN/AuthZ)

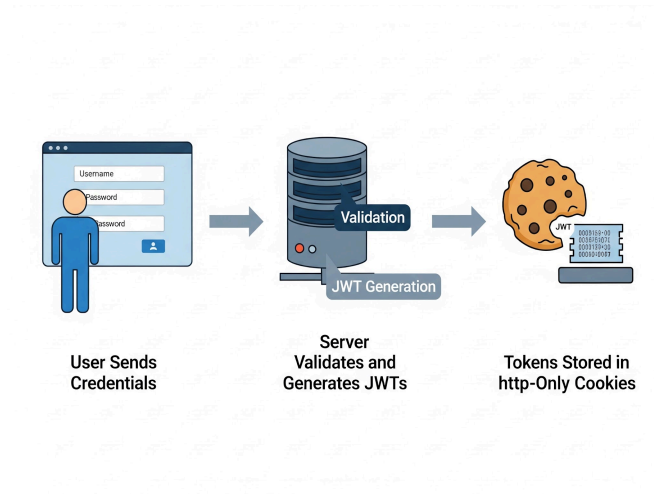
We implement a robust, stateless security model using JSON Web Tokens (JWTs) and Role-Based Access Control (RBAC).

2.1 Secure Login Flow (Step-by-Step)

The login process is designed to be secure and resilient against common attacks.

- User Request:** The **Person** enters their credentials (username/email and password) and sends a request to the server.
- Server Validation:** The backend service receives the request, hashes the received password using **bcrypt**, and compares it to the stored hash in the database.
- Token Generation:** Upon successful validation, the server generates two tokens:
 - Access Token:** Short lifespan (e.g., 15 minutes), used for accessing protected API endpoints.

- **Refresh Token:** Long lifespan (e.g., 7 days), used *only* to obtain a new Access Token.
4. **Token Delivery (Secure):** Both tokens are set in the response headers as **httpOnly cookies**.
- The **httpOnly** flag prevents client-side JavaScript (and thus XSS) from accessing the tokens.
 - The cookies are also secured with the **Secure** flag (only transmitted over HTTPS) and the **SameSite=Strict** flag (for CSRF protection).
5. **Access Protected Data:** The client's subsequent requests automatically include the Access Token cookie, allowing the server to authenticate the user and check their role (**RBAC**).



2.2 Token Lifecycle and Management

Token	Expiry Time	Purpose	Storage
Access Token	Short (e.g., 15 min)	Authenticating API calls	httpOnly Cookie
Refresh Token	Long (e.g., 7 days)	Renewing expired Access Tokens	httpOnly Cookie

Token Renewal Process:

1. When an Access Token expires, an API call fails with a "401 Unauthorized" error.

2. The client then sends a request to a dedicated `/refresh` endpoint, which includes the Refresh Token.
3. The server validates the Refresh Token. If valid, the server issues a new Access Token (and potentially a new Refresh Token), and the cycle continues.
4. If both tokens expire, the user is logged out and must re-authenticate.

3. Password and Identity Security

Requirement	Implementation	Security Benefit
Password Hashing	Use bcrypt with a strong work factor (salting)	Protects passwords against Rainbow Table and brute-force attacks even if the database is compromised.
Strong Policy	Enforce minimum length, complexity (mixed case, numbers, special characters).	Makes brute-forcing individual passwords computationally infeasible.
MFA (Optional)	Support for Time-based One-Time Passwords (TOTP).	Adds a critical second layer of defense against credential theft.

4. Application Programming Interface (API) Security

4.1 Input and Data Validation

- **Schema Validation:** All incoming requests are strictly validated using a library like **Pydantic** (or similar schema validators) to ensure the data adheres to the expected format and type.
- **NoSQL Injection Prevention:** Since we use MongoDB, all data is treated as untrusted. We use MongoDB's drivers/libraries, which automatically parameterize queries, preventing direct execution of malicious user input within the query structure.

4.2 Traffic Control

Mechanism	Implementation Detail	Attack Mitigation
Rate Limiting	Apply limits (e.g., 100 requests per minute per IP) to all endpoints, especially <code>/login</code> .	Prevents brute-force login attempts and Denial of Service (DoS) attacks.

Mechanism	Implementation Detail	Attack Mitigation
CORS Configuration	Strictly limit which domains (origins) can access the API endpoints (e.g., only the official frontend domain).	Prevents malicious external websites from making unauthorized requests to the API.

5. Frontend Security

Vulnerability	Prevention Mechanism	Detail
XSS (Cross-Site Scripting)	Input Sanitization	All user-generated content (e.g., task descriptions) is sanitized on the server before storage and displayed using secure frontend frameworks that automatically escape output.
CSRF (Cross-Site Request Forgery)	CSRF Tokens / SameSite Cookies	We use the <code>SameSite=Strict</code> flag on our cookies, which is the primary defense. If needed, we can also implement explicit CSRF tokens for state-changing requests.
Token Exposure	Avoid localStorage	Tokens are stored exclusively in httpOnly cookies , preventing client-side JavaScript access and XSS exploitation.

6. Data and Transport Protection

- **Data Encryption:** Sensitive PII (e.g., certain user details) and task contents are encrypted at rest using **AES-256** before storage in the database.
- **Transport Security:** All communications between the client, server, and internal components must use **HTTPS** (TLS 1.2+). The Nginx reverse proxy enforces this.

7. Database Security (MongoDB)

1. **Authentication:** MongoDB is configured with strong authentication enabled (username/password/x.509) and runs in an authenticated mode.
2. **Network Isolation:** Access to the database is restricted via a firewall, allowing connections **only** from the application servers' IP addresses (**IP Whitelisting**). No direct public access.
3. **Backups:** A regular, scheduled backup and recovery plan is in place to ensure business continuity. Backups are encrypted and stored off-site.

8. Infrastructure and Environment Security

We adhere to the principle of "Defense in Depth" at the infrastructure layer.

- **Containerization:** The application is deployed using **Docker** containers, which provides process and network isolation.
- **Secure Routing:** An **Nginx Reverse Proxy** is used to handle SSL/TLS termination, secure header configuration (e.g., HSTS), and rate limiting before traffic reaches the application server.
- **Secrets Management:** Sensitive information (database credentials, API keys, encryption keys) are **not** hardcoded. They are injected as **Environment Variables** (or a dedicated secrets manager in a production environment), and access is restricted. The `.env` file must never be committed to source control.

9. Monitoring, Logging, and Error Handling

9.1 Logging and Auditing

The application maintains comprehensive security logs:

- **Audit Logs:** Track all critical user activities (e.g., "Admin created task," "User updated profile") including the timestamp, user ID, and action.
- **Auth Logs:** Log all successful and failed authentication attempts (including IP address). Multiple failures from one IP trigger rate limiting.
- **Monitoring:** Utilize monitoring tools to detect and alert on anomalous behavior (e.g., sudden spikes in failed logins, large data exports).

9.2 Secure Error Handling

- **Generic Responses:** In the event of an error (e.g., API failure, internal server error), the system returns generic error messages to the client (e.g., "An unexpected error occurred").

- **No Exposure:** Never expose internal details like stack traces, system paths, or database connection strings to the end-user. Detailed error information is only written to secure server-side logs.

10. Security Best Practices Summary

Category	Best Practice
Code	Follow the Principle of Least Privilege (PoLP) for both users and system processes.
Dependencies	Regularly update all libraries and frameworks to patch known vulnerabilities.
Configuration	Disable default/unused services and ports (Secure by Default).
Testing	Conduct regular security testing, including penetration testing and vulnerability scanning.
Development	Implement static analysis security testing (SAST) in the continuous integration (CI) pipeline.